Microsoft Teams Calendar Integration

Step-by-Step Backend Integration Guide

WeSearch ATS | For: Backend Engineering Team

Date: October 2026 | API: Microsoft Graph v1.0

Purpose

This document provides a complete, production-ready guide for integrating the Microsoft Graph Calendar API into the WeSearch ATS backend. The integration enables the system to automatically create Microsoft Teams meeting invites when an interview round is scheduled, and to update or cancel those invites on reschedule or cancellation -- without requiring any individual staff member to link their personal Microsoft account.

Architecture Decision

A single shared organizer mailbox (e.g. interviews@wesearch.com) is used for all calendar events. The backend authenticates as an application using the OAuth 2.0 Client Credentials flow -- no user login required. All attendees (candidate, recruiter, client panel) receive calendar invites automatically from this central mailbox.

Option A: Per-User Accounts X	Option B: Single Organiser v (chosen)
X Requires each user to connect their M365 account	v One account set up once by IT admin
X Breaks when staff leave the company	v No dependency on individual staff
X Complex OAuth consent flow per user	v Fully automated -- no user interaction
X Hard to audit or manage centrally	v All invites come from one trackable mailbox

PHASE 1 -- AZURE SETUP

All steps in this phase are performed by your Microsoft 365 / IT Administrator. The backend team needs the four values produced by this phase before writing any code.

Step 1: Register the Application in Azure

The Azure App Registration is the identity of the WeSearch ATS backend in Microsoft's system. Every API call the backend makes to Microsoft Graph is authenticated using this registration.

1. Go to portal.azure.com and sign in as a Global Administrator.
2. In the search bar, type "App registrations" and open it.
3. Click "+ New registration".
4. Fill in the form:
   Name: WeSearch ATS
   Supported account types: "Accounts in this organizational directory only"
   Redirect URI: leave blank
5. Click Register.
6. On the Overview page that opens, copy and save these two values immediately:

Value	Where to find it	Save as env var
Application (client) ID	Overview page -- top section	AZURE_CLIENT_ID
Directory (tenant) ID	Overview page -- top section	AZURE_TENANT_ID

Step 2: Create a Client Secret

The client secret is the password the backend uses to authenticate. It is only shown once at creation time -- copy it immediately.

1. Inside the App Registration, click "Certificates & secrets" in the left menu.
2. Click "+ New client secret".
3. Set description: ats-backend-prod
4. Set expiry: 24 months (recommended -- set a reminder to rotate before expiry).
5. Click Add.
6. In the table that appears, copy the VALUE column (not the Secret ID).
7. Save it immediately as AZURE_CLIENT_SECRET -- it will never be shown again.

IMPORTANT If you navigate away before copying the secret value, it cannot be retrieved. You will need to delete it and create a new one.

Step 3: Add API Permission (Calendars.ReadWrite)

This permission allows the backend application to create, update, and delete calendar events on behalf of the organizer account -- without any user signing in.

1. Inside the App Registration, click "API permissions" in the left menu.
2. Click "+ Add a permission".
3. Select "Microsoft Graph".
4. Select "Application permissions" (NOT Delegated -- this is important).

5. In the search box type: Calendars.ReadWrite
6. Tick the checkbox next to "Calendars.ReadWrite".
7. Click "Add permissions".
8. Back on the API permissions page, click "Grant admin consent for [your org]".
9. Click Yes to confirm.
10. The status column should now show a green tick: "Granted for [your org]".

REQUIRED Granting admin consent requires a Global Administrator or Cloud Application Administrator role. Without this step, every API call will return a 403 Forbidden error.

Step 4: Exchange RBAC -- Required for App-Only Calendar Access

Microsoft requires one additional PowerShell configuration step for application-level calendar access. This is separate from the Azure portal steps above and must be run in Exchange Online PowerShell by someone with Exchange Administrator rights.

REQUIRED This step is mandatory. Without it, the Graph API will return 403 even with Calendars.ReadWrite permission granted. This is a Microsoft security requirement for app-only (non-user) calendar access.

Install the Exchange Online module if not already installed:

PowerShell -- Install module

```
Install-Module -Name ExchangeOnlineManagement -Force
```

Connect and configure:

PowerShell -- Exchange RBAC setup

```
# Connect to Exchange Online (will prompt for admin login)
Connect-ExchangeOnline

# Grant the app permission to access the organizer mailbox
New-ServicePrincipalAuthorization `
    -Identity 'interviews@wesearch.com' `
    -AppId '<AZURE_CLIENT_ID>' `
    -Role 'Application Calendars.ReadWrite'

# Verify the configuration
Test-ServicePrincipalAuthorization `
    -Identity '<AZURE_CLIENT_ID>' `
    -Resource 'interviews@wesearch.com'
```

Replace interviews@wesearch.com with your chosen organizer mailbox address (see Step 5), and replace <AZURE_CLIENT_ID> with the value saved in Step 1.

Step 5: Create the Organizer Mailbox and Get Its Object ID

All Teams meeting invites will be sent from a single shared mailbox. This is a one-time setup. No human logs into this account daily -- it exists purely as the 'sender' identity for all interview invites from WeSearch ATS.

- Recommended address: interviews@wesearch.com
- Type: Regular licensed Microsoft 365 mailbox (must have an Exchange Online license -- Teams meeting generation requires an active mailbox with Teams enabled).
- Shared mailboxes may work but a licensed user account is more reliable for Teams meeting creation.

To get the Object ID of the organizer account:

1. Go to portal.azure.com ? Azure Active Directory ? Users.
2. Search for and open the organizer account (e.g. interviews@wesearch.com).
3. On the Profile page, copy the "Object ID" field.
4. Save it as AZURE_ORGANIZER_USER_ID.

Summary -- four values the backend team needs from Phase 1:

Environment Variable	Description	Example format
AZURE_TENANT_ID	Directory / Tenant ID from Azure AD	xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
AZURE_CLIENT_ID	Application (client) ID	xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
AZURE_CLIENT_SECRET	Client secret value	abc~XYZ123...
AZURE_ORGANIZER_USER_ID	Object ID of interviews@ mailbox	xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx

PHASE 2 -- BACKEND CODE -- SPRING BOOT / JAVA

All steps in this phase are performed by the backend engineering team. The four environment variable values from Phase 1 must be available before starting.

Step 6: Add Maven Dependencies

Add both libraries to pom.xml. The Microsoft Graph SDK handles all API calls; the Azure Identity library handles the OAuth 2.0 Client Credentials authentication flow.

pom.xml

```
<!-- Microsoft Graph SDK v6 (latest stable) -->
<dependency>
    <groupId>com.microsoft.graph</groupId>
    <artifactId>microsoft-graph</artifactId>
    <version>6.26.0</version>
</dependency>

<!-- Azure Identity -- Client Credentials auth provider -->
<dependency>
    <groupId>com.azure</groupId>
    <artifactId>azure-identity</artifactId>
    <version>1.13.3</version>
</dependency>
```

Step 7: Add Environment Variables and application.yml Config

Add the four values from Phase 1 to the backend .env file:

backend/.env

```
# Microsoft Graph / Azure -- Teams Calendar Integration
AZURE_TENANT_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
AZURE_CLIENT_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
AZURE_CLIENT_SECRET=your-client-secret-value-here
AZURE_ORGANIZER_USER_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

Map them in application.yml:

src/main/resources/application.yml

```
app:
  azure:
    tenant-id: ${AZURE_TENANT_ID}
    client-id: ${AZURE_CLIENT_ID}
    client-secret: ${AZURE_CLIENT_SECRET}
    organizer-user-id: ${AZURE_ORGANIZER_USER_ID}
```

Step 8: Create the AzureProperties Configuration Class

AzureProperties.java

```
package com.wesearch.ats.config;
```

```
import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "app.azure")
public record AzureProperties(
    String tenantId,
    String clientId,
    String clientSecret,
    String organizerUserId
) {}
```

Enable it in your main application class or a @Configuration class:

AtsApplication.java

```
@SpringBootApplication
@EnableConfigurationProperties(AzureProperties.class)
public class AtsApplication { ... }
```

Step 9: Create the Graph Client Spring Bean

This bean creates a single authenticated Graph client that is shared across the application. The ClientSecretCredential handles automatic token refresh -- no manual token management needed.

GraphClientConfig.java

```
package com.wesearch.ats.config;

import com.azure.identity.ClientSecretCredential;
import com.azure.identity.ClientSecretCredentialBuilder;
import com.microsoft.graph.authentication.TokenCredentialAuthProvider;
import com.microsoft.graph.requests.GraphServiceClient;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.util.List;

@Configuration
public class GraphClientConfig {

    @Bean
    public GraphServiceClient<?> graphServiceClient(AzureProperties props) {
        ClientSecretCredential credential = new ClientSecretCredentialBuilder()
            .tenantId(props.tenantId())
            .clientId(props.clientId())
            .clientSecret(props.clientSecret())
            .build();

        TokenCredentialAuthProvider authProvider = new TokenCredentialAuthProvider(
            List.of("https://graph.microsoft.com/.default"), credential);

        return GraphServiceClient.builder()
            .authenticationProvider(authProvider)
            .buildClient();
    }
}
```

Step 10: Create the TeamsMeetingService

This service contains three operations: create a meeting (called when an interview round is scheduled), reschedule (called when date/time changes), and cancel (called when round is cancelled). The Teams join URL never changes on reschedule -- the same link works for the updated time.

TeamsMeetingService.java -- Part 1 of 2

```
package com.wesearch.ats.interview;

import com.microsoft.graph.models.*;
import com.microsoft.graph.requests.GraphServiceClient;
import com.wesearch.ats.config.AzureProperties;
import org.springframework.stereotype.Service;
import java.util.LinkedList;
import java.util.List;
```

```
@Service
public class TeamsMeetingService {

    private final GraphServiceClient<?> graphClient;
    private final String organizerUserId;

    public TeamsMeetingService(GraphServiceClient<?> graphClient,
                               AzureProperties props) {
        this.graphClient = graphClient;
        this.organizerUserId = props.organizerUserId();
    }

    /**
     * Creates a Teams-enabled calendar event.
     * @param subject      Meeting subject line
     * @param bodyHtml     HTML body of the invite email
     * @param startDateTime ISO datetime e.g. '2026-10-10T10:00:00'
     * @param endDateTime  ISO datetime e.g. '2026-10-10T11:00:00'
     * @param timeZone     e.g. 'India Standard Time'
     * @param attendeeEmails all invitee email addresses
     * @return MeetingResult with teamsEventId + joinUrl
     */
    public MeetingResult createMeeting(String subject, String bodyHtml,
            String startDateTime, String endDateTime,
            String timeZone, List<String> attendeeEmails) {

        Event event = new Event();
        event.subject = subject;

        ItemBody body = new ItemBody();
        body.contentType = BodyType.HTML;
        body.content = bodyHtml;
        event.body = body;

        DateTimeTimeZone start = new DateTimeTimeZone();
        start.dateTime = startDateTime;
        start.timeZone = timeZone;
        event.start = start;

        DateTimeTimeZone end = new DateTimeTimeZone();
        end.dateTime = endDateTime;
        end.timeZone = timeZone;
        event.end = end;

        LinkedList<Attendee> attendees = new LinkedList<>();
        for (String email : attendeeEmails) {
            Attendee a = new Attendee();
            EmailAddress ea = new EmailAddress();
            ea.address = email;
            a.emailAddress = ea;
            a.type = AttendeeType.REQUIRED;
            attendees.add(a);
        }
        event.attendees = attendees;
```

```
        // This is what generates the Teams meeting link
        event.isOnlineMeeting = true;
        event.onlineMeetingProvider = OnlineMeetingProviderType.TEAMS_FOR_BUSINESS;

        Event created = graphClient
            .users(organizerUserId)
            .calendar()
            .events()
            .buildRequest()
            .post(event);

        String joinUrl = (created.onlineMeeting != null)
            ? created.onlineMeeting.joinUrl : null;

        return new MeetingResult(created.id, joinUrl);
    }
```

TeamsMeetingService.java -- Part 2 of 2

```
    /** Reschedule: updates start/end. Join URL stays the same -- do not update meeting_link. */
    public void rescheduleMeeting(String teamsEventId,
                                  String newStartDateTime,
                                  String newEndDateTime,
                                  String timeZone) {
        Event patch = new Event();

        DateTimeTimeZone start = new DateTimeTimeZone();
        start.dateTime = newStartDateTime;
        start.timeZone = timeZone;
        patch.start = start;

        DateTimeTimeZone end = new DateTimeTimeZone();
        end.dateTime = newEndDateTime;
        end.timeZone = timeZone;
        patch.end = end;

        graphClient.users(organizerUserId)
            .events(teamsEventId)
            .buildRequest()
            .patch(patch);
        // Microsoft automatically sends update notification to all attendees
    }

    /** Cancel: deletes the calendar event. Microsoft sends cancellation to all attendees. */
    public void cancelMeeting(String teamsEventId) {
        graphClient.users(organizerUserId)
            .events(teamsEventId)
            .buildRequest()
            .delete();
    }

    public record MeetingResult(String teamsEventId, String joinUrl) {}
}
```

Step 11: Database Migration

Add two columns to the interview_rounds table. Both are nullable -- if the Graph API call fails, the round is still saved and can be retried.

SQL Migration

```
ALTER TABLE interview_rounds
    ADD COLUMN meeting_link    VARCHAR(512),
    ADD COLUMN teams_event_id  VARCHAR(255);
```

Column	Type	Stores	Used for
meeting_link	VARCHAR(512)	onlineMeeting.joinUrl from Graph response	Displayed to users as Teams join link
teams_event_id	VARCHAR(255)	Event id from Graph response	Reschedule and cancel API calls

Step 12: Wire Into Interview Round Endpoints

Call TeamsMeetingService from the three relevant interview round operations:

InterviewRoundService.java -- relevant excerpts

```
// ?? CREATE interview round ??????????????????????????????????????????
String subject = String.format("[%s] %s Interview -- %s | %s (%s)",
    round.getRoundType(), candidateName, jobTitle, clientName);

String bodyHtml = "<p>This interview has been scheduled via WeSearch ATS.<br>" +
    "Please use the Teams link to join at the scheduled time.</p>";

MeetingResult meeting = teamsMeetingService.createMeeting(
    subject, bodyHtml,
    round.getScheduledDate() + "T" + round.getScheduledTime() + ":00",
    round.getScheduledDate() + "T" + round.getScheduledEndTime() + ":00",
    "India Standard Time",
    round.getInviteeEmails()   // includes candidate, recruiter, panel
);
round.setMeetingLink(meeting.joinUrl());
round.setTeamsEventId(meeting.teamsEventId());

// ?? RESCHEDULE ???????????????????????????????????????????????????????
teamsMeetingService.rescheduleMeeting(
    round.getTeamsEventId(),
    newStartDateTime, newEndDateTime,
    "India Standard Time"
);
// meeting_link stays the same -- do NOT overwrite it

// ?? CANCEL ???????????????????????????????????????????????????????????
teamsMeetingService.cancelMeeting(round.getTeamsEventId());
```

PHASE 3 -- API REFERENCE AND EXPECTED RESPONSES

Reference section for testing and debugging the integration.

Step 13: Graph API Request -- Create Event

The backend sends this HTTP request to Microsoft Graph. The SDK handles token acquisition and serialization automatically.

HTTP Request

```
POST https://graph.microsoft.com/v1.0/users/{AZURE_ORGANIZER_USER_ID}/calendar/events
Authorization: Bearer {access_token}   <- handled automatically by the SDK
Content-Type: application/json

{
  "subject": "L1 Interview -- Arjun Mehta | Full Stack Developer (PWC)",
  "isOnlineMeeting": true,
  "onlineMeetingProvider": "teamsForBusiness",
  "start": { "dateTime": "2026-10-15T10:00:00", "timeZone": "India Standard Time" },
  "end":   { "dateTime": "2026-10-15T11:00:00", "timeZone": "India Standard Time" },
  "attendees": [
    { "emailAddress": { "address": "arjun.mehta@gmail.com" },  "type": "required" },
    { "emailAddress": { "address": "recruiter@wesearch.com" }, "type": "required" },
    { "emailAddress": { "address": "panel@pwc.com" },          "type": "required" }
  ],
  "body": { "contentType": "HTML",
    "content": "<p>Interview scheduled via WeSearch ATS.</p>" }
}
```

Graph API returns HTTP 201 Created with this response (shortened):

HTTP 201 Response

```
{
  "id": "AAMkAGViNDU7zAAAAA7zAAAZe6CkAAA=",   <- save as teams_event_id
  "isOnlineMeeting": true,
  "onlineMeetingProvider": "teamsForBusiness",
  "onlineMeeting": {
    "joinUrl": "https://teams.microsoft.com/l/meetup-join/19%3ameeting_...",
    "conferenceId": "177513992",
    "tollNumber": "+91 22 4718 3956"
  },
  "start": { "dateTime": "2026-10-15T10:00:00.0000000", ... },
  ...
}
```

NOTE Store event.id as teams_event_id and onlineMeeting.joinUrl as meeting_link in the database. The joinUrl is permanent -- it does not change on reschedule. Only teams_event_id is needed for future PATCH (reschedule) and DELETE (cancel) calls.

Step 14: Timezone Reference for India

Use the Windows timezone identifier for India Standard Time. This is the value to pass to the timeZone field in all API calls:

Timezone values

```
timeZone: "India Standard Time"   // IST, UTC+5:30

// Other common timezones if needed:
// "UTC"
// "Pacific Standard Time"   // PST, UTC-8
// "Eastern Standard Time"   // EST, UTC-5
```

Step 15: Error Handling Guidance

HTTP Status	Likely cause	Fix
400 Bad Request	Missing required field or invalid datetime format	Check subject, start, end, attendees
401 Unauthorized	Invalid or expired client secret	Regenerate client secret in Azure portal
403 Forbidden	Admin consent not granted OR Exchange RBAC mi	Complete Steps 3 and 4
404 Not Found	Wrong organizer user ID, or user does not exist	Verify AZURE_ORGANIZER_USER_ID
429 Too Many Req.	Graph API rate limit hit	Implement exponential backoff retry
503 Service Unavail	Microsoft Graph outage	Retry with backoff; surface error to user

IMPORTANT The most common setup error is 403. It almost always means either (a) admin consent was not granted in the Azure portal, or (b) the Exchange RBAC PowerShell step (Step 4) was skipped. Check both before investigating other causes.

PHASE 4 -- SUMMARY AND HANDOFF CHECKLIST

Use this checklist to track completion of each item before going to production.

Microsoft 365 / IT Admin Tasks

[ ]	**Step 1**	Register Azure App -- obtain Client ID + Tenant ID
[ ]	**Step 2**	Create client secret -- save value immediately
[ ]	**Step 3**	Add Calendars.ReadWrite application permission + grant admin consent
[ ]	**Step 4**	Run Exchange RBAC PowerShell for the organizer mailbox
[ ]	**Step 5**	Create organizer mailbox (interviews@wesearch.com) + get its Object ID
[ ]	**Step 5**	Share all 4 env var values securely with backend team

Backend Engineering Team Tasks

[ ]	**Step 6**	Add microsoft-graph and azure-identity Maven dependencies
[ ]	**Step 7**	Add 4 env vars to .env + map in application.yml
[ ]	**Step 8**	Create AzureProperties @ConfigurationProperties record
[ ]	**Step 9**	Create GraphClientConfig @Bean
[ ]	**Step 10**	Create TeamsMeetingService with create / reschedule / cancel methods
[ ]	**Step 11**	Run DB migration -- add meeting_link + teams_event_id columns
[ ]	**Step 12**	Wire TeamsMeetingService into interview round create/reschedule/cancel
[ ]	**--**	Test: create an interview round and verify invite arrives in attendee inbox
[ ]	**--**	Test: reschedule -- verify attendees receive update notification
[ ]	**--**	Test: cancel -- verify attendees receive cancellation
[ ]	**--**	Frontend: zero changes needed -- meetingLink field already exists

Official Reference Documentation

- **Microsoft Graph -- Create online meeting via Calendar API**
  `https://learn.microsoft.com/en-us/graph/outlook-calendar-online-meetings`
- **Microsoft Graph -- POST /users/{id}/calendar/events**
  `https://learn.microsoft.com/en-us/graph/api/calendar-post-events?view=graph-rest-1.0`
- **Microsoft Graph -- Event resource type**
  `https://learn.microsoft.com/en-us/graph/api/resources/event?view=graph-rest-1.0`
- **Microsoft Graph Java SDK (GitHub)**
  `https://github.com/microsoftgraph/msgraph-sdk-java`
- **Azure Identity Java SDK**
  `https://learn.microsoft.com/en-us/java/api/overview/azure/identity-readme`
- **Exchange RBAC -- App-only calendar access**
  `https://learn.microsoft.com/en-us/graph/auth-limit-mailbox-access`